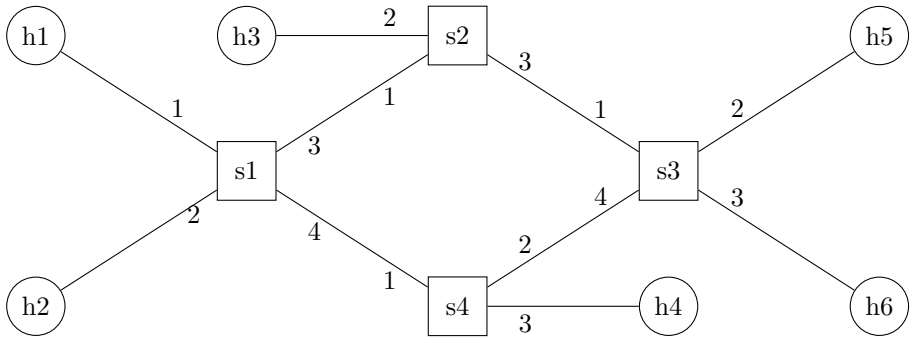
Software Defined Networking – 22 luglio 2019

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Nelle risposte usare per ogni pacchetto il seguente formato:

Nodo → interfaccia: MAC sorgente → MAC destinazione; ethertype; IP sorgente → IP destinazione; TTL; protocollo; porta sorgente → porta destinazione;

Negli switch sono preinstallate le regole in Figura 1.

L'host h3 porta 1234 apre una connessione TCP verso h5 porta 80.

(a) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

> **Soluzione:** s2 → 3,1: MAC3 → MAC5; IPv4; IP1 → IP5; TTL:200; TCP; 1234 → 80;

(b) Dire quanti pacchetti arrivano in h5 e che valore ha il campo TTL.

> **Soluzione:** Uno con TTL=200.

L'host h2 porta 3322 invia un datagramma UDP verso h5 porta 123.

(c) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

> **Soluzione:**
> `packet_in`(MAC2 → MAC5; IPv4; IP2 → IP5; TTL:200; UDP; 3322 → 123)
> Il controllore risponde installando due regole, la prima delle quali è eseguita immediatamente
> .
> MAC2 → MAC5; IPv4; IP2 → IP5; TTL:200; UDP; 3322 → 1147

(d) Dire quanti pacchetti arrivano in h5 e con che TTL.

> **Soluzione:** Uno con TTL=200.

(e) L'host h5 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

> **Soluzione:** In s3 il pacchetto viene inviato al controllore, che non risponde. Pertanto non arrivano pacchetti in s1.

(f) Scrivere quanti pacchetti arrivano in h2 e che valore ha il campo TTL.

Soluzione: Non arrivano pacchetti in h2.

L'host h5 porta 7777 apre una connessione TCP verso l'host h3 porta 245.

(g) Scrivere una regola da inserire in s3 che permetta a tutti e soli i pacchetti della connessione di raggiungere la destinazione.

Soluzione:

Switch s4

Priority	Match	Action
5	eth_type=IPv4 ip_src=IP5 ip_dst=IP3 ip_proto=TCP tcp_src=7777 tcp_dst=245	Output(1)

(h) Scrivere una regola da inserire in s3 che permetta a tutti e soli i pacchetti di risposta di raggiungere la destinazione.

Soluzione:

Switch s4

Priority	Match	Action
5	eth_type=IPv4 ip_src=IP3 ip_dst=IP5 ip_proto=TCP tcp_src=245 tcp_dst=777	Output(2)

Figura 1: Regole iniziali

Switch s1

Priority	Match	Action
0	*	Output(FLOOD)
5	eth_src = MAC1	Output(CONTROLLER,128)
5	eth_src = MAC2	Output(CONTROLLER,128)

Switch s2

Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = IPv4 ip_proto = TCP	Output(FLOOD)

Switch s3

Priority	Match	Action
0	*	Output(FLOOD)
5	eth_src = MAC5	Output(CONTROLLER,128)
5	eth_src = MAC6	Output(CONTROLLER,128)

Switch s4

Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = IPv4 ip_proto = UDP	Output(FLOOD)

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (datapath.id == 1 and pkt_udp is not None and pkt_udp.dst_port < 1024):
        match = parser.OFPMatch(eth_type=ETH_TYPE_IP, ipv4_dst = pkt_ipv4.dst,
                ip_proto=IPPROTO_UDP, udp_dst=pkt_udp.dst_port)
        actions = [ parser.OFPSetField(udp_dst = pkt_udp.dst_port + 1024),
                parser.OFPActionOutput(4)]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=10,
            match=match, instructions=inst, buffer_id=buffer_id)
        datapath.send_msg(mod)

        match = parser.OFPMatch(eth_type=ETH_TYPE_IP, ipv4_dst = pkt_ipv4.src,
                ip_proto=IPPROTO_UDP, udp_src=pkt_udp.dst_port + 1024)
        actions = [ parser.OFPSetField(udp_src = pkt_udp.dst_port),
                parser.OFPActionOutput(FLOOD)]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=10,
            match=match, instructions=inst)
        datapath.send_msg(mod)

        return
```

Esercizio 2

Si consideri un sistema di misura del traffico basato su *consistent hashing*. Siano date le funzion hash h, con output di 128 bit e g, con output di $m = 16$ bit. Ad ogni pacchetto è associato un *flow id* F tale che connessioni TCP distinte abbiano flow id distinti. Sia L la lunghezza del pacchetto in ottetti.

In ogni nodo è presente un array A di 2^m elementi che viene azzerato ogni $T = 1$ ora. Se $h(F)$ è minore di una soglia R, il nodo incrementa incrementa di L l'elemento dell'array $A[g(F)]$. Se, dopo l'incremento, $A[g(F)]$ è maggiore di una soglia S, il *flow id* F è salvato su disco.

Ad uno di questi nodi arrivano pacchetti a velocità $C = 1\,\text{Gbit/s}$.

(a) Calcolare la soglia R in modo da filtrare una connessione ogni 128.

Soluzione:

$$R = \frac{2^{128}}{128} = 2^{121}$$

(b) Calcolare la soglia S in modo da salvare i flussi da almeno 100 Mbit.

Soluzione:

$$S = \frac{100 \cdot 10^6\,\text{Mbit}}{8} = 12{,}5 \cdot 10^6\,\text{ottetti}$$

(c) Dopo un'ora di misure, sono stati salvati $n = 10$ *flow id*. Dare una stima del numero di connessioni che hanno portato almeno 100 Mbit di traffico.

Soluzione: Considerando che viene campionata una connessione ogni 128 la stima è $10 \times 128 = 1280$

(d) Ad un certo punto, sono arrivate al sistema $n = 10000$ connessioni da 50 Mbit. Calcolare la probabilità che il *flow id* del prossimo pacchetto sia salvato su disco. Considerare il caso pessimo.

Soluzione: Nel caso pessimo, le n connessioni sono tutte campionate e si dispongono su 5000 elementi di A. La probabilità è:

$$\frac{1}{128}\frac{5000}{2^{16}} = 6 \cdot 10^{-4}$$

(e) In un progetto si sceglie di usare come funzione g i primi m bit di h. Dire se e come cambia la probabilità calcolata al punto precedente.

Soluzione:

Per le connessioni che sono campionate, i primi 7 bit di $h(F)$ sono tutti 0, quindi gli elementi di A effettivamente usati sono solo 2^9 anziché 2^{16}.

Nel caso pessimo, le n connessioni occupano tutti gli elementi di A. La probabilità è:

$$\frac{1}{128} = 7{,}8 \cdot 10^{-3}$$

Esercizio 3

Si consideri un'interfaccia gestita da uno scheduler Deficit Round Robin. La velocità di trasmissione è 8 Gbit/s. Al tempo $t = 0$ sono presenti i seguenti pacchetti, di cui viene indicata la lunghezza in byte, ordinati dal più recente al più vecchio:

Coda 1: 50, 175, 50

Coda 2: 25, 75, 150

Coda 3: 100, 150, 100

Coda 4: 50

Al tempo $t = 400$ ns si presenta alla coda 4 un pacchetto di lunghezza 100.

(a) Si considerino quanti di lunghezza 200. Simulare l'algoritmo DRR, indicando il tempo al quale termina ogni trasmissione ed il deficit residuo.

Soluzione:

Round	Coda	Deficit iniziale	Deficit finale	Pacchetto trasmesso	Tempo fine tx
1	1	0	150	50	50
	2	0	50	150	200
	3	0	10	100	300
	4	0	0	50	350
2	1	150	0	175+50	575
	2	50	0	75+25	675
	3	100	0	150+100	925
	4	0	0	100	1025

(b) Si consideri ora un quanto di lunghezza 50. Dire quando inizia la trasmissione il secondo pacchetto della coda 4.

Soluzione:
Al tempo 825 ns.

Si consideri uno scenario con lunghezza massima dei pacchetti 1000 byte.

(c) Calcolare il valore minimo del quanto tale che sia sempre trasmesso almeno un pacchetto per ogni coda visitata e tale che le code 2, 3 e 4 abbiano collettivamente la stessa banda della coda 1.

Soluzione: Dobbiamo rispettare il vincolo $q_2 + q_3 + q_4 = q_1$. Qualsiasi soluzione con $q_2 = q_3 = q_4$ e con $q_1 = 3q_2$ va bene. Per trasmettere almeno un pacchetto per ogni coda ad ogni round fissiamo $q_2 = q_3 = q_4 = 1000$ e $q_1 = 3000$.

(d) Usando il valore del quanto calcolato al punto precedente, calcolare il ritardo massimo prima di iniziare la trasmissione che deve aspettare un pacchetto che si presenti alla coda 4 trovandola vuota.

Soluzione:
Il caso pessimo si ha quando abbiamo 1 pacchetto di lunghezza massima in trasmissione e per le altre code un deficit pari alla lunghezza massima di un pacchetto meno 1 più un intero quanto.

$$D_{\max} = L + (q_1 + L - 1) + (q_2 + L - 1) + (q_3 + L - 1) \simeq 9\,\mu\text{s}$$

Esercizio 4

Un sistema di controllo degli accessi conserva una black list di indirizzi MAC cui è vietato l'accesso in rete. La black list usa una hash table di 256 elementi. Si consideri la funzione hash h usata per indicizzare la tabella.

Si consideri di aver effettuato l'inserimento di 300 indirizzi MAC.

(a) Qual è la lunghezza (in bit) dell'output della funzione h.

Soluzione: $\log_2 256 = 8$

(b) Calcolare la probabilità che vi sia almeno una collisione.

Soluzione: Poiché $300 > 256$, c'è sicuramente una collisione (principio dei cassetti).

(c) Calcolare il numero medio di indirizzi MAC per elemento della tabella.

Soluzione: $300/256 = 1{,}17$

(d) Calcolare la probabilità di falso positivo.

Soluzione: Le hash table non danno falsi positivi.

(e) Dare una stima dell'occupazione di memoria della tabella. Considerare puntatori di 64 bit.

Soluzione:

$$256 \times 64 + 300 \times (64 + 48) = 49\,984\,\text{byte}$$

Si consideri ora la blacklist implementata con un filtro di Bloom costituito da un array di 1024 bit e 4 funzioni hash

(f) Calcolare la probabilità di falso positivo.

Soluzione:

$$(1 - \exp(-4 \times 300/1024))^4 = 23\%$$